# Supply Chain Monitoring and Optimization Platform

# Week 5 - Deploy & Automate with Azure DevOps

## 1. Objective

The purpose of Week 5 is to automate the execution of our supply chain monitoring scripts using **Azure DevOps Pipelines**. This ensures continuous integration and deployment (CI/CD) practices where Python scripts can be executed automatically upon changes in the repository.

### Deliverables:

1. Azure DevOps YAML pipeline file
2. Execution log or console output of pipeline run

## 2. Prerequisites

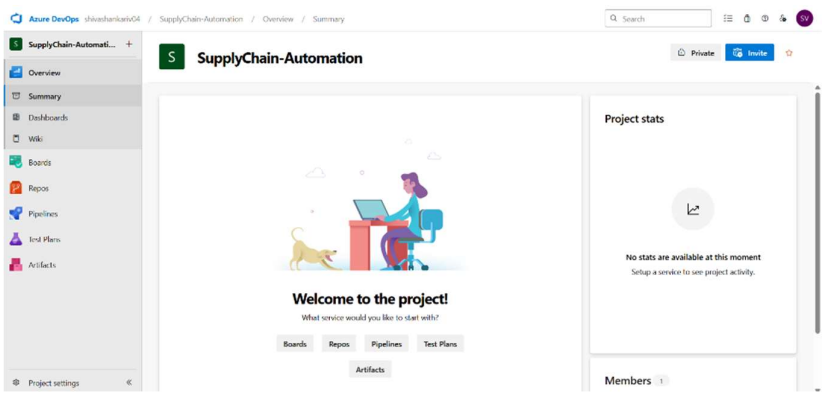
Before creating and running the pipeline, the following are required:

1. **Azure DevOps Account** – A valid subscription with access to Azure DevOps Projects.
2. **Code Repository** – Source code stored in Azure Repos or an integrated GitHub repository.
3. **Project Files**:
    - Python scripts (e.g., Supply_Pipeline.py)
    - Dependency list (requirements.txt)
    - Dataset files (if needed, uploaded with the repository)

## 3. Step-by-Step Process

### Step 1: Create Azure DevOps Project

1. Sign in to Azure DevOps.
2. Create a new **DevOps Project** (e.g., *SupplyChain-Automation*).



3. Connect your repository (Azure Repos or GitHub).

**Step 2: Prepare Project Repository**

1. Push the following to your repository root:
    - Supply_Pipeline.py (main Python script).
    - requirements.txt (list of Python dependencies such as pandas, numpy, pyspark).
    - Supporting files (CSV datasets if required).
2. Confirm repository structure:

    / (root)

    ├── Supply_Pipeline.py

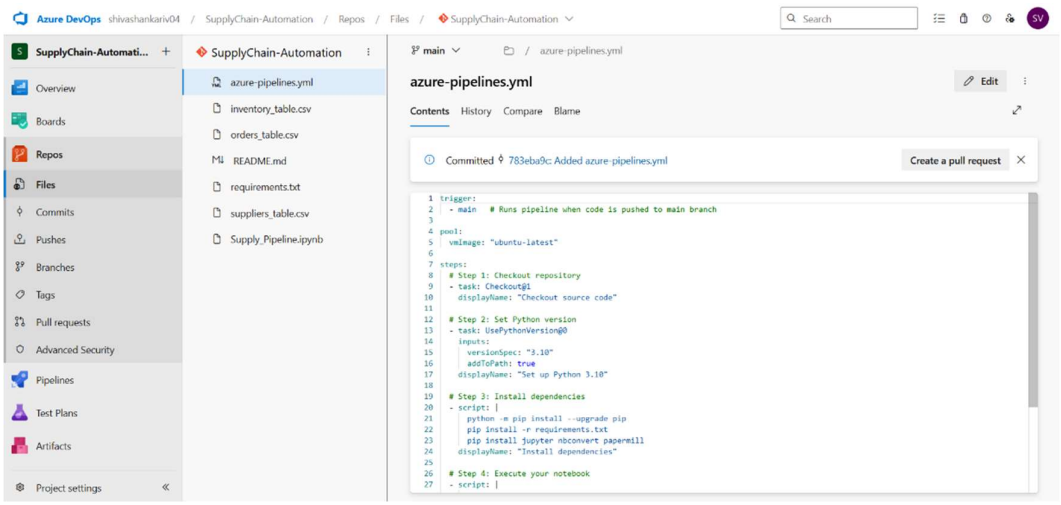
    ├── requirements.txt

    ├── inventory.csv

    ├── orders.csv

    ├── suppliers.csv

    └── azure-pipelines.yml  (pipeline definition)




**Step 3: Define the Pipeline (YAML)**

Create a YAML pipeline file named azure-pipelines.yml in the repository root

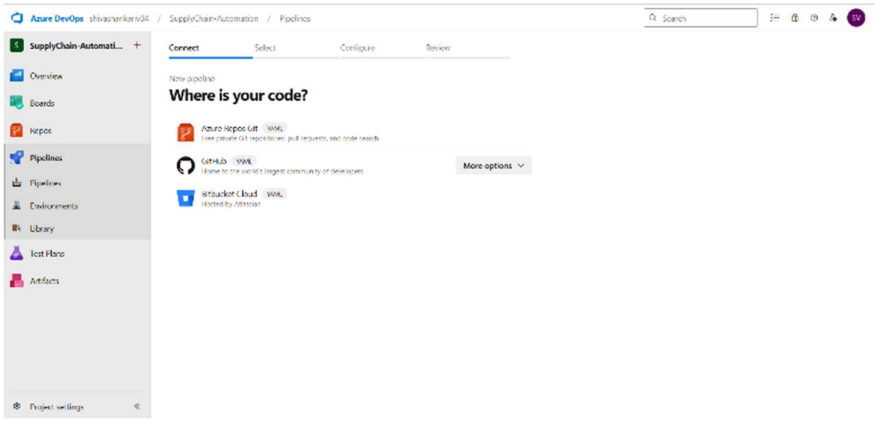
This pipeline performs:

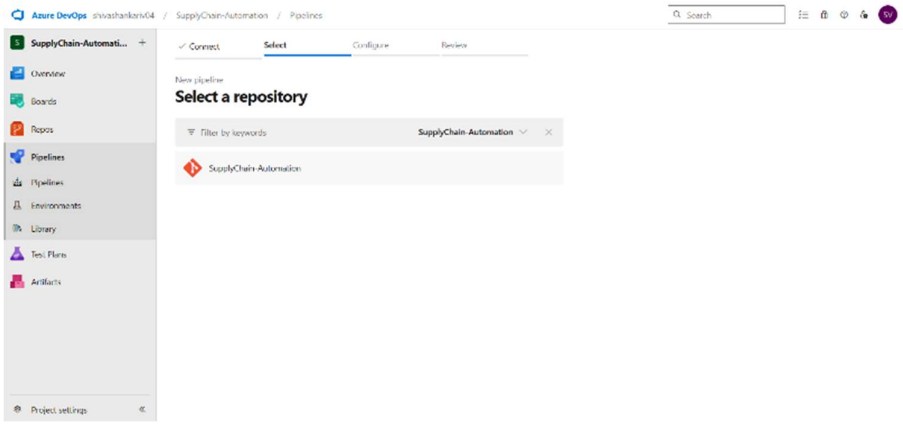
1. **Trigger:** Runs whenever code is pushed to the main branch.
2. **Setup:** Uses latest Ubuntu VM with Python 3.10.
3. **Install:** Installs required libraries from requirements.txt.

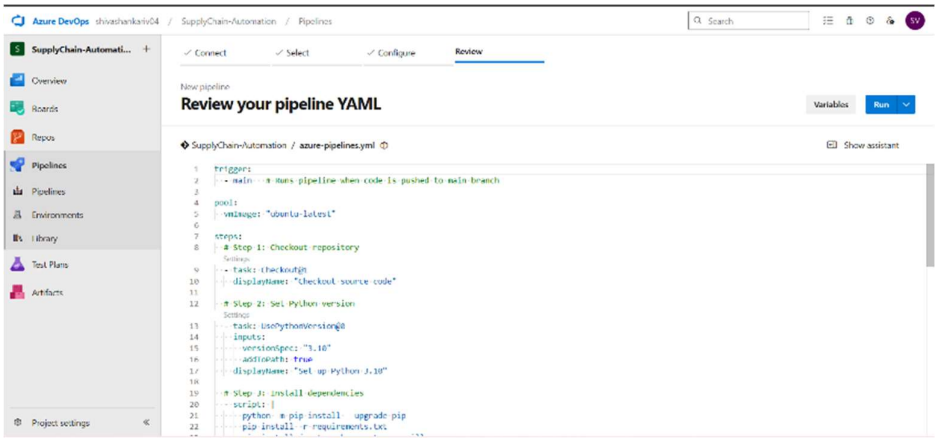
4. **Execution:** Runs the project's Python script.

### Step 4: Run the Pipeline

1. Navigate to **Pipelines > New Pipeline**.
2. Select repository and choose **YAML file** (azure-pipelines.yml).
3. Run the pipeline manually or wait for an automatic trigger on commit.
4. Monitor execution in the Azure DevOps console.



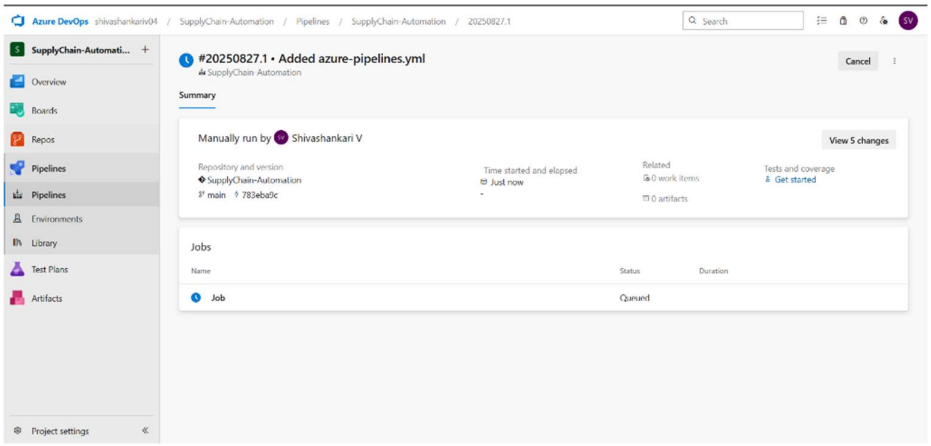



**Step 5: Validate & Collect Deliverables**

1. Ensure pipeline completes successfully.
2. Download **execution logs** showing:
    - Dependency installation output.
    - Script execution results (delay detection, filtering, grouping, etc.).

3. Document the successful run as evidence for Week 5 submission.




**4. Example Execution Log (Sample Snippet)**

Starting: Run Supply Chain Script

==========

Requirement already satisfied: pandas in /usr/local/lib/python3.10/site-packages

Requirement already satisfied: numpy in /usr/local/lib/python3.10/site-packages

...

>>> Running run_pipeline.py

Total Orders: 10

Delayed Orders: 7

Percentage Delayed: 70.0%

Stock Summary Exported Successfully

==========

Pipeline completed successfully

## 5. Conclusion

In Week 5, the project is deployed on Azure DevOps using CI/CD principles. By configuring a YAML pipeline, we automated:

- Installation of dependencies
- Execution of supply chain monitoring scripts
- Logging and verification of results

This approach ensures repeatability, transparency, and scalability in executing supply chain analytics as part of the overall capstone project.